Data Structures (CSN12101)
ASSIGNMENT-6

Name : Aayansh Niranjan Date :11/03/2026
Reg_No.: 20253004

Q1. Your task is to write a C program that accepts two polynomials as input, performs the addition, and outputs the resulting polynomial.

A1. CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct node{
   int coef, exp;
   struct node* next;
};

struct node* create(){
   struct node *head=NULL,*temp,*new;
   int n,c,e,i;
   printf("No. of terms: ");
   scanf("%d",&n);
   for(i=0;i<n;i++){
      new=(struct node*)malloc(sizeof(struct node));
      scanf("%d%d",&c,&e);
      new->coef=c; new->exp=e; new->next=NULL;
      if(head==NULL) head=temp=new;
      else{ temp->next=new; temp=new; }
   }
   return head;
}

void display(struct node* p){
   while(p){
      printf("%dx^%d",p->coef,p->exp);
      if(p->next) printf(" + ");
      p=p->next;
   }
}

struct node* add(struct node* p1,struct node* p2){
   struct node *res=NULL,*temp=NULL,*new;
   while(p1 && p2){
      new=(struct node*)malloc(sizeof(struct node));
      new->next=NULL;
      if(p1->exp==p2->exp){
         new->coef=p1->coef+p2->coef;
         new->exp=p1->exp;
         p1=p1->next; p2=p2->next;
      }
      else if(p1->exp>p2->exp){
         new->coef=p1->coef; new->exp=p1->exp;
         p1=p1->next;
      }
```

```
        else{
            new->coef=p2->coef; new->exp=p2->exp;
            p2=p2->next;
        }

        if(!res) res=temp=new;
        else{ temp->next=new; temp=new; }
    }
    return res;
}

int main(){
    struct node *p1,*p2,*sum;
    printf("Enter polynomial 1 (coef exp):\n");
    p1=create();
    printf("Enter polynomial 2 (coef exp):\n");
    p2=create();
    sum=add(p1,p2);
    printf("Result: ");
    display(sum);
}
```

OUTPUT:

```
Enter polynomial 1 (coef exp):
No. of terms: 2
6 3
5 2
Enter polynomial 2 (coef exp):
No. of terms: 2
7 3
4 2
Result: 13x^3 + 9x^2(base) User@cctf-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Aayansh_20253004/Data_structures/Assignment_6$
```

Q2. Suppose you are working in a software company that specializes in mathematical computations. Your current task is to develop a program that can perform polynomial addition. Polynomials are algebraic expressions consisting of variables and coefficients. You need to create a program where the user can input two polynomials, and the program will return their multiplication.

A2. CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct node{
   int coef,exp;
   struct node* next;
};
struct node* insert(struct node* head,int c,int e){
   struct node *new=(struct node*)malloc(sizeof(struct node));
   new->coef=c; new->exp=e; new->next=head;
   return new;
}
struct node* create(){
   int n,c,e,i;
   struct node* head=NULL;
   printf("No. of terms: ");
   scanf("%d",&n);
   for(i=0;i<n;i++){
      scanf("%d%d",&c,&e);
      head=insert(head,c,e);
   }
   return head;
}
void display(struct node* p){
   while(p){
      printf("%dx^%d",p->coef,p->exp);
      if(p->next) printf(" + ");
      p=p->next;
   }
}
struct node* multiply(struct node* p1,struct node* p2){
   struct node *res=NULL;
   struct node *a,*b;
   for(a=p1;a;a=a->next)
      for(b=p2;b;b=b->next)
         res=insert(res,a->coef*b->coef,a->exp+b->exp);
   return res;
}
int main(){
   struct node *p1,*p2,*mul;
   printf("Enter polynomial 1 (coef exp):\n");
   p1=create();
   printf("Enter polynomial 2 (coef exp):\n");
   p2=create();
   mul=multiply(p1,p2);
   printf("Result: ");
   display(mul);
   printf("\n");
}
```

OUTPUT:

```
(base) User@cctf-HP-Pro-Tower-400-G9-PCI-Desktop-PC:~/Documents/Aayansh_20253004/Data_structures/Assignment_6$ ./p2
Enter polynomial 1 (coef exp):
No. of terms: 2
4 2
3 1
Enter polynomial 2 (coef exp):
No. of terms: 2
5 2
6 1
Result: 20x^4 + 24x^3 + 15x^3 + 18x^2
```

Q3. SWAP to implement Stack using an array:
a. Push an element onto a stack. b. Pop the stack
c. Display the contents of the stack.
A3. CODE:

```
#include<stdio.h>
#define MAX 100
int stack[MAX],top=-1;
void push(int x){
    if(top==MAX-1) printf("Stack Overflow\n");
    else stack[++top]=x;
}
void pop(){
    if(top==-1) printf("Stack Underflow\n");
    else printf("Popped: %d\n",stack[top--]);
}
void display(){
    int i;
    if(top==-1) printf("Stack Empty\n");
    else{
        for(i=top;i>=0;i--) printf("%d ",stack[i]);
        printf("\n");
    }
}
int main(){
    int ch,x;
    while(1){
        printf("\n1.Push 2.Pop 3.Display 4.Exit\n");
        scanf("%d",&ch);
        switch(ch){
            case 1:scanf("%d",&x);push(x);break;
            case 2:pop();break;
            case 3:display();break;
            case 4:return 0;
        }
    }
    printf("\n");
}
```

OUTPUT:

```
aayansh207@aayansh-PC:/media/aayansh207/Extra/Projects_Main/MNNIT/SEMESTER-2/Data_structures/Assignment_6$ ./p3

1.Push 2.Pop 3.Display 4.Exit
1
10

1.Push 2.Pop 3.Display 4.Exit
1
20

1.Push 2.Pop 3.Display 4.Exit
1
30

1.Push 2.Pop 3.Display 4.Exit
3
30 20 10

1.Push 2.Pop 3.Display 4.Exit
2
Popped: 30

1.Push 2.Pop 3.Display 4.Exit
3
20 10
```

Q4.WAP to implement Stack using linked list:
a. Push an element onto a stack. b. Pop the stack
c. Display the contents of the stack.
A4. CODE:

```
#include<stdio.h>
#include<stdlib.h>
struct node{
    int data;
    struct node*next;
};
struct node*top=NULL;
void push(int x){
    struct node*temp=malloc(sizeof(struct node));
    temp->data=x;
    temp->next=top;
    top=temp;
}
void pop(){
    if(top==NULL){printf("Stack Empty\n");return;}
    struct node*temp=top;
    printf("Popped: %d\n",temp->data);
    top=top->next;
    free(temp);
}
void display(){
    struct node*temp=top;
    while(temp){
        printf("%d ",temp->data);
        temp=temp->next;
    }
    printf("\n");
}
int main(){
    int ch,x;
    while(1){
        printf("\n1.Push 2.Pop 3.Display 4.Exit\n");
        scanf("%d",&ch);
        switch(ch){
            case 1:scanf("%d",&x);push(x);break;
            case 2:pop();break;
            case 3:display();break;
            case 4:return 0;
        }
    }
    printf("\n");
}
```

OUTPUT:

```
aayansh207@aayansh-PC:/media/aayansh207/Extra/Projects_Main/MNNIT/SEMESTER-2/Data_structures/Assignment_6$ ./p4

1.Push 2.Pop 3.Display 4.Exit
1
5

1.Push 2.Pop 3.Display 4.Exit
1
15

1.Push 2.Pop 3.Display 4.Exit
3
15 5

1.Push 2.Pop 3.Display 4.Exit
2
Popped: 15

1.Push 2.Pop 3.Display 4.Exit
3
5
```

Q5. SWAP to implement Queue using an array:
a. Insert an element into the queue. b. Delete an element from the queue.
c. Display the contents of the queue.
A5. CODE:

```
#include<stdio.h>
#define MAX 100
int q[MAX],front=-1,rear=-1;
void insert(int x){
   if(rear==MAX-1) printf("Queue Full\n");
   else{
      if(front==-1) front=0;
      q[++rear]=x;
   }
}
void del(){
   if(front==-1||front>rear) printf("Queue Empty\n");
   else printf("Deleted: %d\n",q[front++]);
}
void display(){
   int i;
   if(front==-1||front>rear) printf("Queue Empty\n");
   else{
      for(i=front;i<=rear;i++) printf("%d ",q[i]);
      printf("\n");
   }
}
int main(){
   int ch,x;
   while(1){
      printf("\n1.Insert 2.Delete 3.Display 4.Exit\n");
      scanf("%d",&ch);
      switch(ch){
         case 1:scanf("%d",&x);insert(x);break;
         case 2:del();break;
         case 3:display();break;
         case 4:return 0;
      }
   }
   printf("\n");
}
```

OUTPUT:

```
aayansh207@aayansh-PC:/media/aayansh207/Extra/Projects_Main/MNNIT/SEMESTER-2/Data_structures/Asstgnment_6$ ./p5

1.Insert 2.Delete 3.Display 4.Exit
1
10

1.Insert 2.Delete 3.Display 4.Exit
1
20

1.Insert 2.Delete 3.Display 4.Exit
1
30

1.Insert 2.Delete 3.Display 4.Exit
3
10 20 30

1.Insert 2.Delete 3.Display 4.Exit
2
Deleted: 10

1.Insert 2.Delete 3.Display 4.Exit
3
20 30
```

Q6. WAP to implement Queue using linked list:
a. Insert an element into the queue. b. Delete an element from the queue.
c. Display the contents of the queue.
A6. CODE:

```
#include<stdio.h>
#include<stdlib.h>
struct node{
   int data;
   struct node*next;
};
struct node*front=NULL,*rear=NULL;
void insert(int x){
   struct node*temp=malloc(sizeof(struct node));
   temp->data=x;
   temp->next=NULL;
   if(rear==NULL){
      front=rear=temp;
      return;
   }
   rear->next=temp;
   rear=temp;
}
void del(){
   if(front==NULL){printf("Queue Empty\n");return;}
   struct node*temp=front;
   printf("Deleted: %d\n",temp->data);
   front=front->next;
   if(front==NULL) rear=NULL;
   free(temp);
}
void display(){
   struct node*temp=front;
   while(temp){
      printf("%d ",temp->data);
      temp=temp->next;
   }
   printf("\n");
}
int main(){
   int ch,x;
   while(1){
      printf("\n1.Insert 2.Delete 3.Display 4.Exit\n");
      scanf("%d",&ch);
      switch(ch){
         case 1:scanf("%d",&x);insert(x);break;
         case 2:del();break;
         case 3:display();break;
         case 4:return 0;
      }
   }
   printf("\n");
}
```

OUTPUT:

```
aayansh207@aayansh-PC:/media/aayansh207/Extra/Projects_Main/MNNIT/SEMESTER-2/Data_structures/Assignment_6$ ./p6

1.Insert 2.Delete 3.Display 4.Exit
1
7

1.Insert 2.Delete 3.Display 4.Exit
1
14

1.Insert 2.Delete 3.Display 4.Exit
1
21

1.Insert 2.Delete 3.Display 4.Exit
3
7 14 21
```